Tema 2: Servicios y Protocolos de Aplicaciones

2.1. Introducción a las aplicaciones de red

2.2. Navegación Web

2.3. Servicio de nombres de dominio

2.4. Correo electrónico

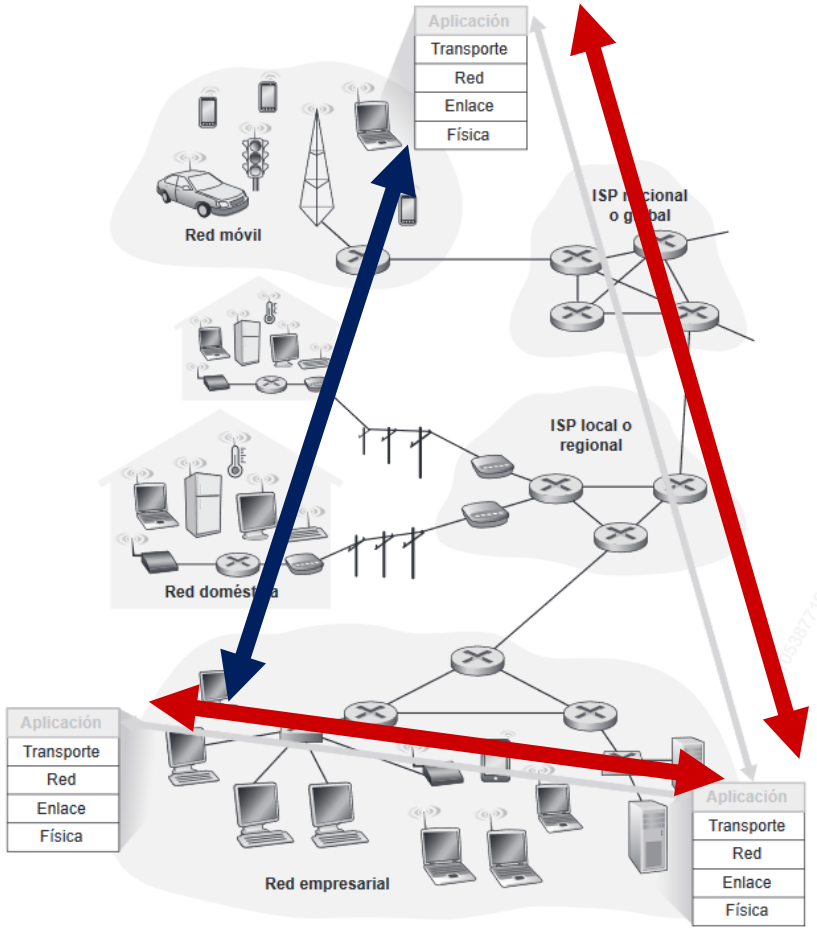
2.6. DHCP

2.6. Arquitectura P2P

2.7. Aplicaciones multimedia

2.1. Introducción a la aplicaciones de red

- Crear una aplicación de red (por ejemplo una aplicación web) implica escribir programas que:
  - ✓ se ejecuten en distintos sistemas finales (el navegador del usuario y el sw del servidor web)
  - ✓ Se comuniquen entre si a través de la red
- Los dispositivos de interconexión de red no ejecutan aplicaciones de usuario
- Para el desarrollador, la arquitectura de red es fija y proporciona servicios a las aplicaciones
- Dos posibles **arquitecturas de aplicación**:
  - ✓ Cliente-Servidor
  - ✓ Peer-to-Peer (P2P)



(Fuente: Redes de computadoras. Un enfoque descendente 7ED. James F. Kurose, Keith W. Rose. 2017)

Arquitectura Cliente-Servidor

- Servidores
  - ✓ Host activo siempre
  - ✓ Dirección IP fija
  - ✓ Se pueden agrupar en centros de datos, escalabilidad
  - ✓ Dan servicio a las solicitudes de muchos host
- Clientes
  - ✓ Pueden conectarse de forma intermitente
  - ✓ Pueden tener dirección IP dinámica
  - ✓ Se comunican con el servidor
  - ✓ No se comunican entre ellos

Arquitectura P2P

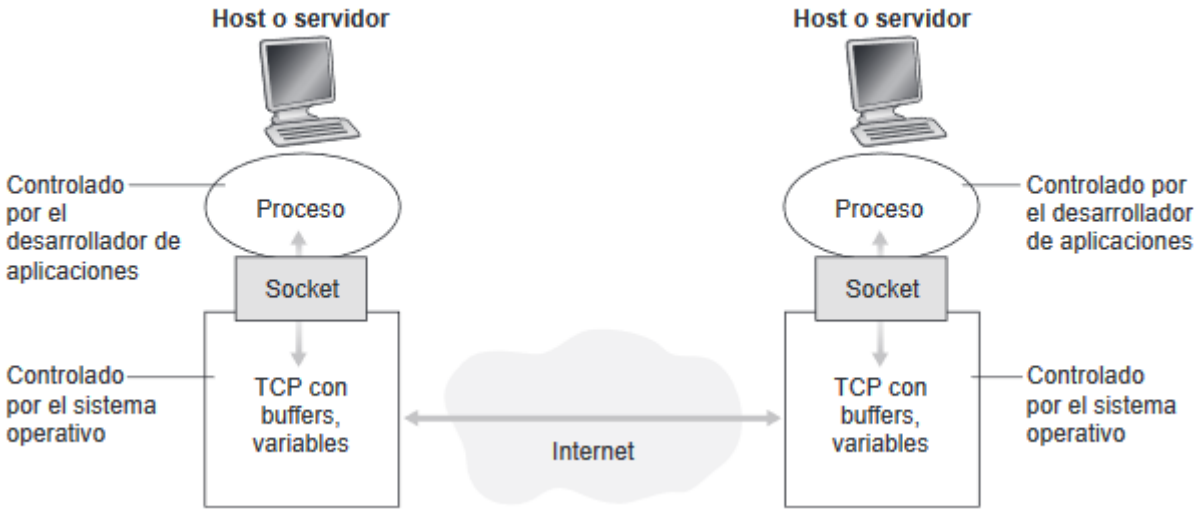
- Comunicación directa entre dos host (pares – ***peers***)
- Conexión intermitente
- Los host (pares) son controlados por los usuarios
- No necesidad de un servidor dedicado -> *arquitectura peer-to-peer*
- Auto-escalable y descentralizada
- Problemas de seguridad, rendimiento y escalabilidad

Comunicación entre procesos

- Proceso: programa que se ejecuta en un host
  - ✓ Dos procesos, en el mismo host, se comunican usando comunicación entre procesos (definido en el SO)
  - ✓ Procesos en distintos host se comunican intercambiando mensajes (a través de la **red**) -> socket
- Arquitectura **cliente-servidor**
  - ✓ Proceso cliente: proceso que inicia la comunicación
  - ✓ Proceso servidor: proceso que espera a ser contactado
- Aquitectura P2P
  - ✓ Cada host tiene procesos cliente y servidor

Sockets

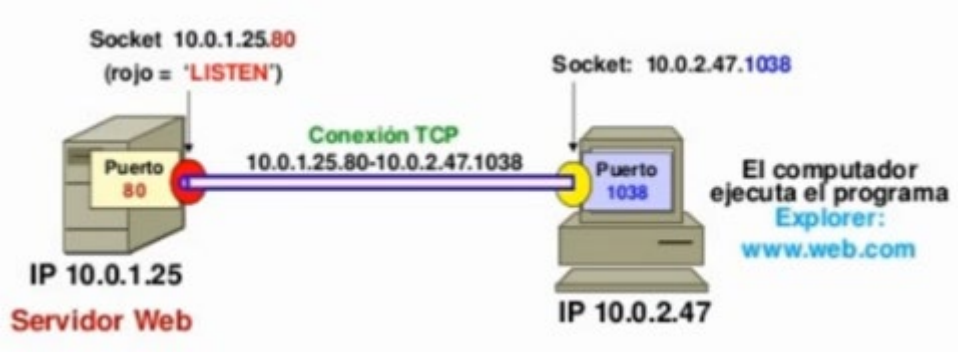
- Interfaz de programación con la que se contruyen las aplicaciones de red
- Interfaz entre la capa de aplicación y la de transporte de un host -> API (interfaz de programación de aplicaciones)
- Desarrollador de aplicaciones
  - ✓ Control total capa de aplicación del socket
  - ✓ Sobre capa de transporte
    - Elección protocolo
    - Algunos parámetros



(Fuente: Redes de computadoras. Un enfoque descendente 7ED. James F. Kurose, Keith W. Rose. 2017)

Direccionamiento

- Necesidad de identificar el proceso receptor
  - ✓ Dirección de host -> Dirección IP
  - ✓ Identificador de proceso (socket) en el host destino -> Número de puerto
    - Ejemplos:
      - ➢ HTTP server: 80
      - ➢ mail server: 25
- Para enviar un mensaje HTTP al servidor web de esei.uvigo.es:
  - ✓ Dirección IP : 217.76.159.23
  - ✓ Número de puerto: 80



Servicios de transporte

¿Qué servicios de la capa de transporte necesitan las aplicaciones?

- Integridad de datos (entrega correcta y completa)
  - ✓ Muchas apps (e.j., correo electrónico, transferencia de ficheros, transferencia documentos web) no soportan pérdida de datos y necesitan garantizar el 100% de fiabilidad en la transferencia de datos
  - ✓ Otras apps (e.j., audio/video en tiempo real) toleran pérdidas de datos
- Rendimiento (garantizar tasa de transferencia a una velocidad especificada)
  - ✓ Algunas apps (e.j., multimedia) son sensibles al ancho de banda y tienen requisitos específicos de tasa de trasferencia
  - ✓ Otras aplicaciones (denominadas “ aplicaciones elásticas”, e.j., correo electrónico, transf. ficheros) hacen uso de la tasa de transferencia disponible
- Temporización (retardo en la entrega)
  - ✓ Apps en tiempo real (e.j., telefonía IP, juegos multijugador) requieren mínimos retardos en la entrega de datos para ser efectivas
- Seguridad
  - ✓ Cifrado, confidencialidad, mecanismo para garantizar la integridad, mecanismos de autenticación

Servicios de transporte de Internet

Servicio TCP:

- Orientado a conexión: intercambio de **información de control** (negociación) entre cliente y servidor antes del intercambio de mensajes
- Transferencia de datos fiable: garantiza entrega de datos enviados sin errores y en el orden correcto
- Control de congestión: regular en envío de datos cuando la red está saturada (servicio para mejorar el funcionamiento general de la red)
- Control de flujo: regular que el emisor no sobrecargue al receptor
- **No proporciona**: temporización, seguridad (salvo con SSL), garantizar de un rendimiento mínimo

Servicio UDP:

- Ligero y simple -> servicios mínimos
- No orientado a conexión
- Transferencia de datos no fiable
- No proporciona: fiabilidad, control de flujo, control de congestión, temporización, seguridad

Protocolos capa de aplicación

Aplicación	Protocolo de la capa de aplicación	Protocolo de transporte subyacente
Correo electrónico	SMTP [RFC 5321]	TCP
Acceso remoto a terminal	Telnet [RFC 854]	TCP
Web	HTTP [RFC 2616]	TCP
Transferencia de archivos	FTP [RFC 959]	TCP
Flujos multimedia	HTTP (p. ej. YouTube)	TCP
Telefonía por Internet	SIP [rfc 3261], RTP [RFC 3550] o propietario (p. ej. Skype)	UDP o TCP

(Fuente: Redes de computadoras. Un enfoque descendente 7ED. James F. Kurose, Keith W. Rose. 2017)

Características

La Web

- Opera bajo demanda
- Es fácil publicar información
- Una **página web** está formada por objetos
  - ✓ Un objeto puede ser un fichero HTML, una imágen JPEG, un applet Java, un fichero de audio,…
- Una página web consiste en un fichero base HTML y varios objetos referenciados
- Cada objeto es referenciable mediante una URL

$$\underbrace{\texttt{https://www.google.com}}_{\text{nombre host}}\underbrace{\texttt{/images/logo.png}}_{\text{nombre ruta}}$$

- Protocolo de nivel de aplicación: HTTP (protocolo de transferencia de hipertexto)

HTTP

Modelo cliente-servidor

- cliente: navegador (Explorer, Firefox, Chrome) que solicita, recibe (usando el protocolo HTTP) y muestra objetos Web
- servidor: el servidor Web (apache, Microsoft Internet Information Server) responde a la peticiones enviando (usando el protocolo HTTP) los objetos

Define

- Estructura mensajes HTTP
- Intercambio de mensajes entre cliente y servidor

♦ Comportamiento solicitud-respuesta de HTTP.

(Fuente: Redes de computadoras. Un enfoque descendente 7ED. James F. Kurose, Keith W. Rose. 2017)

HTTP

- Usa TCP como protocolo de transporte
  - ✓ El cliente inicia una conexión TCP (crea socket) al servidor, puerto 80
  - ✓ El servidor acepta la conexión TCP del cliente
  - ✓ Intercambio de mensajes HTTP entre el navegador (cliente HTTP) y el servidor Web (servidor HTTP)
  - ✓ Cierre de la conexión TCP
- HTTP envía mensajes al interfaz del socket, a partir de ahí lo gestiona TCP (protocolo fiable)
- HTTP no mantiene ninguna información acerca de los clientes -> protocolo sin estado (stateless)

Conexiones HTTP

- Conexiones no persistentes: cada par solicitud/respuesta se envía a través de una conexión TCP
  - ✓ Un objeto para cada conexión TCP (pueden ser en serie)
  - ✓ Después de enviar la respuesta, cierre de la conexión
  - ✓ Descargar múltiples objetos requiere múltiples conexiones
    - Ej. Una página con un archivo base HTML y 10 imágenes JPEG -> 11 conexiones TCP
    - Navegadores abren entre 5-10 conexiones en paralelo (configurable por el usuario)
  - ✓ Pueden sobrecargar al servidor Web
- Conexiones persistentes: todas las solicitudes/respuestas se envían a través de la misma conexión TCP
  - ✓ HTTP emplea conexiones persistentes de manera predeterminada (clientes y servidores configurables para usar no persistentes)
  - ✓ El servidor deja la conexión TCP abierta después de enviar la respuesta
    - Ej. Una página con un archivo base HTML y 10 imágenes JPEG -> 1 conexión TCP
  - ✓ Las solicitudes de objetos se pueden realizar de forma consecutiva, sin esperar cada respuesta

Conexiones HTTP

Ejemplo conexión no persistente URL: http:// di.uvigo.es/index.html

1a. El cliente HTTP inicia una conexión TCP al proceso en el servidor HTTP di.uvigo.es en el puerto 80

1b. El servidor HTTP en el host di.uvigo.es, que esta esperando conexiones TCP en el puerto 80, acepta la conexión y lo notifica al cliente

2. El cliente HTTP envía un *mensaje de solicitud* (con la URL) en la conexión TCP. El mensaje indica que el cliente quiere el objeto index.html

3. El servidor HTTP recibe el mensaje de solicitud, crea el *mensaje de respuesta* con el objeto solicitado, y envia el mensaje a su socket

4. El servidor HTTP cierra la conexión TCP

5. El cliente HTTP recibe el mensaje de respuesta con el fichero html y muestra html. Al analizar el fichero html, se encuentra referencias a 3 objetos imagen

time

6. Los pasos 1-5 se repite para cada objeto imagen

Formato mensajes HTTP

Dos tipos de mensajes HTTP: solicitud, respuesta

- Mensaje de solicitud HTTP
  - ✓ ASCII

Linea solicitud (GET, POST, comandos HEAD)

carácter intro

carácter linea en blanco

```
GET /index.html HTTP/1.1\r\n
Host: www-net.cs.umass.edu\r\n
User-Agent: Firefox/3.6.10\r\n
Accept: text/html,application/xhtml+xml\r\n
Accept-Language: en-us,en;q=0.5\r\n
Accept-Encoding: gzip,deflate\r\n
Accept-Charset: ISO-8859-1,utf-8;q=0.7\r\n
Keep-Alive: 115\r\n
Connection: keep-alive\r\n
\r\n
```

Lineas cabecera

Fin de la cabecera

Formato mensajes HTTP

- Mensaje de respuesta HTTP
  - ✓ ASCII

línea de estado (versión protocolo, código de estado, mensaje explicativo estado)

líneas de cabecera

Cuerpo, datos, por ej. fichero html solicitado

```
HTTP/1.1 200 OK\r\n
Date: Sun, 26 Sep 2010 20:09:20 GMT\r\n
Server: Apache/2.0.52 (CentOS)\r\n
Last-Modified: Tue, 30 Oct 2007 17:00:02
   GMT\r\n
ETag: "17dc6-a5c-bf716880"\r\n
Accept-Ranges: bytes\r\n
Content-Length: 2652\r\n
Keep-Alive: timeout=10, max=100\r\n
Connection: Keep-Alive\r\n
Content-Type: text/html; charset=ISO-8859-
   1\r\n
\r\n
data data data data data ...
```

Servicio DNS

Identificación de host:

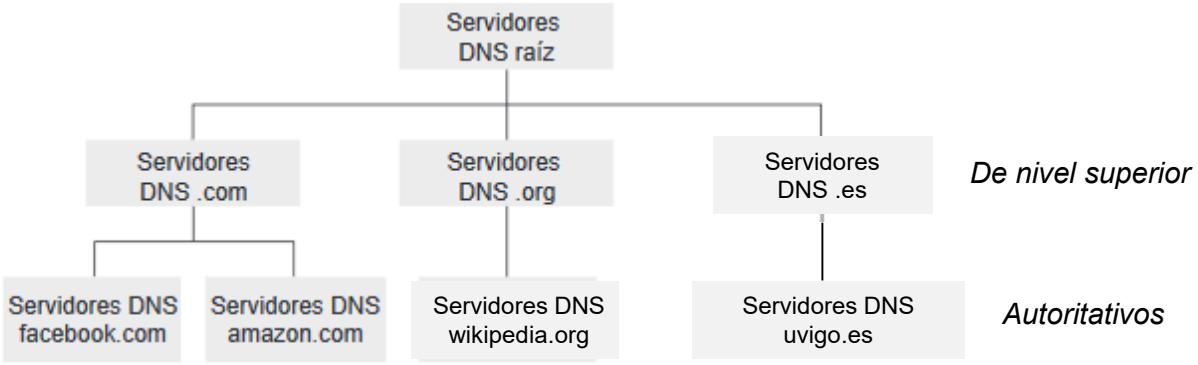
- Dirección IP (4 bytes, 32 bit): usado para identificar el dispositivo en la red (193.146.32.200)
  - ✓ Estructura jerárquica
  - ✓ Informa de la ubicación del dispositivo en la red
- Nombre: entendidos por las personas(www.uvigo.es)
  - ✓ No proporcionan información, excepto el código de país

¿Cómo establecer la correspondencia entre ambos? DNS (Domain Name Server)

- BD distribuida implementada en una jerarquía de servidores DNS
- Protocolo de la capa de aplicación que realiza la traducción de nombres a dirección IP (consulta BD)
  - ✓ UDP, puerto 53
  - ✓ Usado por HTTP y SMTP
    - La consulta DNS añade retardo a la aplicación -> Importancia de la **cache DNS**

2.3. Servicio de nombre de dominio

Funcionamiento DNS



(Fuente: Redes de computadoras. Un enfoque descendente 7ED. James F. Kurose, Keith W. Rose. 2017)

Ejemplo: un cliente quiere obtener la IP para *www.uvigo.es*:

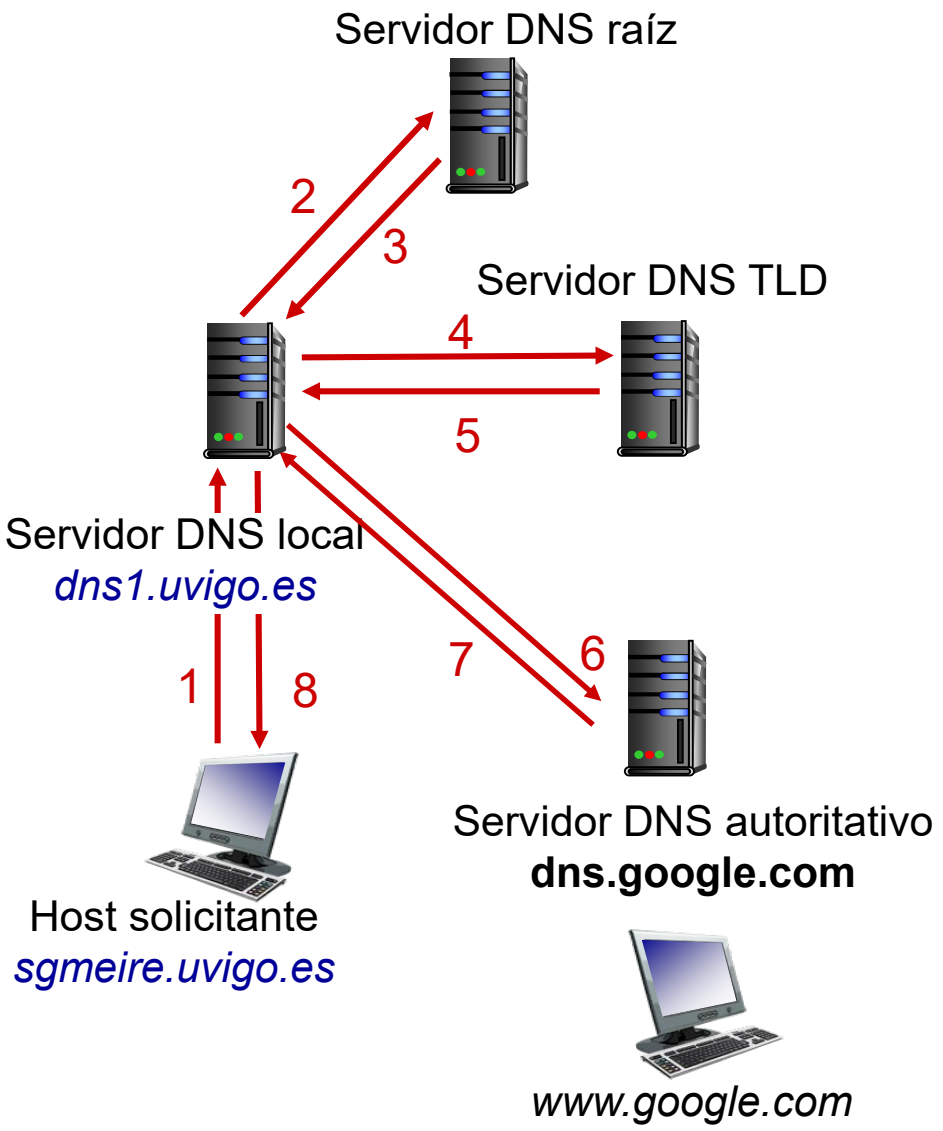
- el cliente consulta al servidor raíz para encontrar el servidor DNS *.es*
- el raíz redirige la consulta al servidor DNS .es para obtener el servidor DNS *uvigo.es*
- el servidor .es consulta al servidor DNS de uvigo.es para obtener la dirección IP de *www.uvigo.es*

2.3. Servicio de nombre de dominio

Servidor DNS local

El DNS local:

- no pertenece estrictamente a la jerarquía de servidores de nombres
- cada ISP (ISP residencial, empresa, universidad) tiene uno: "servidor de nombres predeterminado"
- cuando el host realiza una consulta DNS, la consulta se envía a su servidor DNS local
  - ✓ tiene caché local de pares de traducción de nombre a dirección recientes (¡pero puede estar desactualizado!)
  - ✓ actúa como proxy, reenvía la consulta a la jerarquía

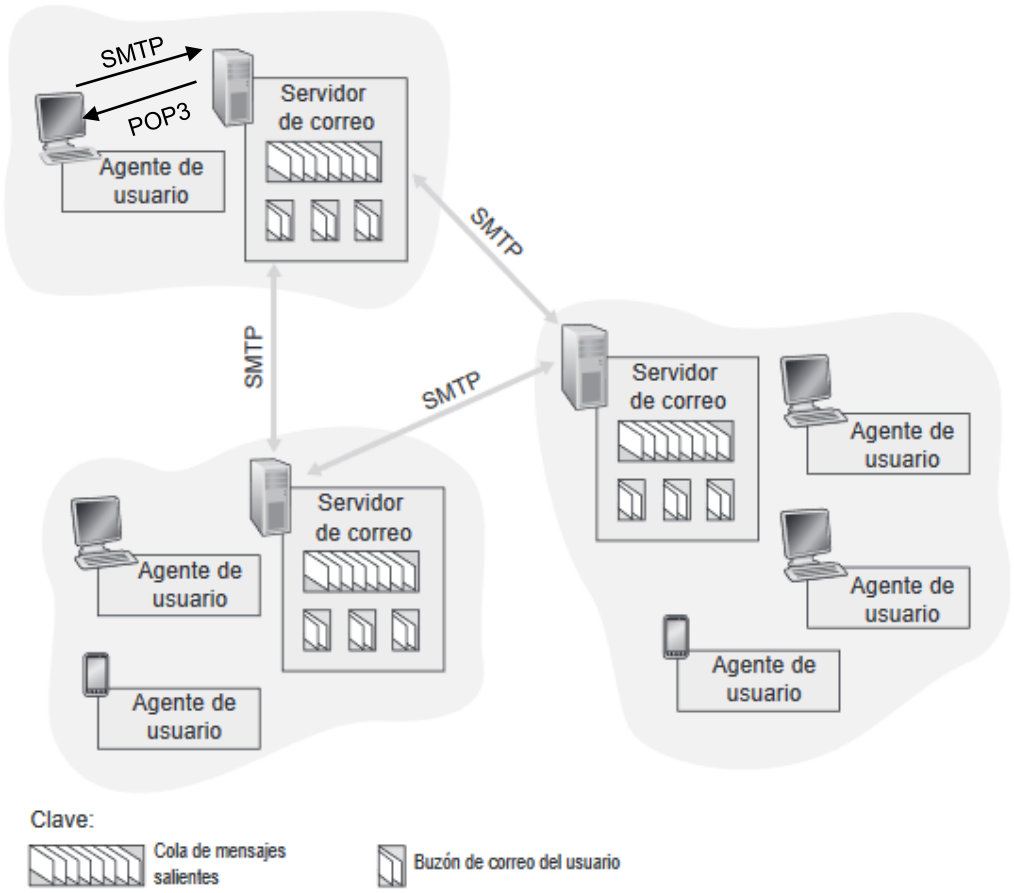


Fuente: Powerpoints slides copyright 1996-2016 J.F. Kurose, K.W. Ross

2.4. Correo electrónico

Componentes

- Medio de comunicación asíncrono
- Tan antiguo como Internet
- Usa varios protocolos
- Tres componentes principales
  - ✓ Cliente de correo (user agent): compone, edita y lee correos electrónicos
    - Lector de correo (Outlook, Thunderbird, iPhone mail)
    - Los mensajes entrantes o salientes se almacenan en el servidor
  - ✓ Servidor de correo (mail server): almacena los mensajes de correo entrantes y salientes
    - Núcleo de la infraestructura de correo electrónico
  - ✓ Protocolo de transferencia de correo (SMTP)
- Protocolos de descarga: POP3, IMAP, HTTP



(Fuente: Redes de computadoras. Un enfoque descendente 7ED. James F. Kurose, Keith W. Rose. 2017)

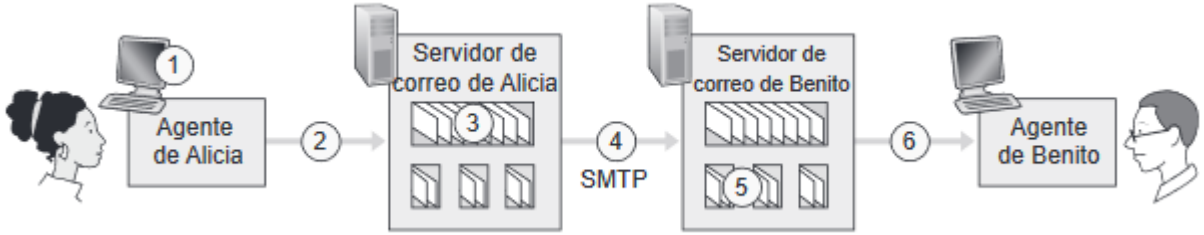
SMTP

- Principal protocolo de la capa de aplicación para el correo electrónico [RFC 5321]
- usa TCP para transferir mensajes de correo electrónico de manera fiable del cliente al servidor, puerto 25
- transferencia directa: servidor emisor al servidor receptor
- tres fases de transferencia
  - ✓ saludo
  - ✓ transferencia de mensajes
  - ✓ cierre
- interacción comando / respuesta (como HTTP)
  - ✓ comandos: texto ASCII
  - ✓ respuesta: código de estado y texto
- los mensajes deben estar en ASCII de 7 bits
- Usa conexiones persistentes

SMTP

Funcionamiento básico:

1. El usuario origen compone mediante su *cliente de correo* un mensaje dirigido a la dirección de correo del usuario destino.
2. El cliente de correo envía el mensaje mediante SMTP al servidor de correo del usuario origen, que lo sitúa en la cola de mensajes salientes.
3. El cliente SMTP abre una conexión TCP con el servidor de correo del usuario destino.
4. El cliente SMTP envía el mensaje sobre la conexión TCP.
5. El servidor de correo del usuario destino ubica el mensaje en el *buzón de correo* del usuario destino.
6. El usuario destino invoca a su *cliente de correo* para leer el mensaje utilizando POP3, IMAP o HTTP



(Fuente: Redes de computadoras. Un enfoque descendente 7ED. James F. Kurose, Keith W. Rose. 2017)

Protocolos de acceso

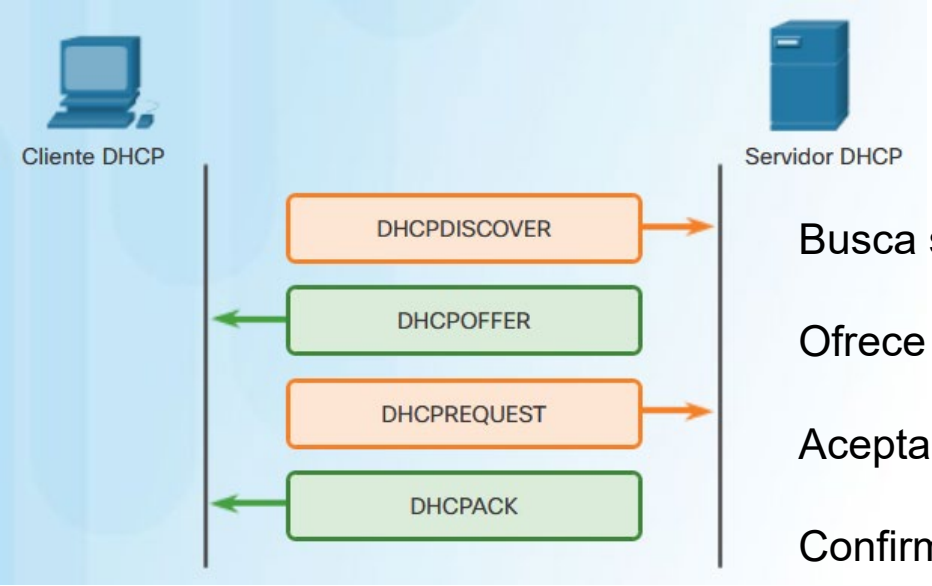
SMTP: entrega / almacenamiento al servidor del receptor (protocolo de inserción, *push*)

Protocolo de acceso al correo: recuperación del servidor (protocolo de extracción, *pull*)

- POP3: Protocolo de la oficina postal [RFC 1939]
  - ✓ Se inicia cuando el agente cliente abre una conexión TCP con el servidor de correo, puerto 110
  - ✓ Tres fases:
    - Autorización: nombre de usuario y contraseña (autenticar al usuario)
    - Transacción: el cliente de correo recupera los mensajes
    - Actualización: finalizada la sesión POP3 el servidor elimina los mensajes marcados para borrado (modo descargar y borrar) o los mantiene (modo descargar y mantener)
  - ✓ Usa conexiones sin estado entre sesiones
- IMAP: Protocolo de acceso al correo de Internet [RFC 1730]
  - ✓ Mejora a POP3
  - ✓ Mantiene los mensajes en el servidor
  - ✓ más funciones: incluida la manipulación de mensajes almacenados en el servidor, mantener información entre sesiones o permitir la descarga de partes de los mensajes
- WebMAIL: accesible desde cualquier cliente con HTTP (Gmail, Hotmail, Yahoo! Correo, etc.)
  - ✓ No utiliza cliente de correo

2.5. DHCP

- Asignación automática de direcciones IP, máscara, Gateway y otra información de red
- Se asignan por un tiempo limitado (una vez expirado se devuelven al grupo)
- Varios dispositivos pueden ser servidores DHCP
  - ✓ Redes grandes: servidor local y dedicado
  - ✓ Redes domésticas: router local
- Funcionamiento



Busca servidor

Ofrece conexión (IP, máscara, Gateway)

Acepta conexión

Confirma

Fuente: cisco.netacad.com

2.6. Arquitecturas P2P

- No dependen de una infraestructura de servidores
- Parejas de host se conectan de manera intermitente y se comunican directamente entre si
- Ejemplos: BitTorrent, Skype, …
- BitTorrent
  - ✓ Protocolo para distribución de archivos
  - ✓ Conjunto de torrents: colección de pares que participan en la distribución de un archivo
  - ✓ Los pares de un torrent descargan fragmentos del mismo tamaño (256 kb) y actualizan con otros pares
  - ✓ Sobre UDP

2.7. Aplicaciones Multimedia

Son aplicaciones que transmiten/reproducen contenido con **múltiples medios**: audio, video, texto, imágenes, etc.

Categorías principales:

- Streaming multimedia
  - ✓ Streaming almacenado/VOD: Netflix, YouTube, Spotify, Amazon Prime
  - ✓ Streaming en vivo: Twitch, YouTube Live, transmisiones deportivas
  - ✓ Streaming conversacional/interactivo: Zoom, Teams, WhatsApp videollamadas, Google Meet
- Telefonía por Internet (VoIP)
  - ✓ Skype, WhatsApp llamadas, Discord voz
  - ✓ Solo audio (pero también multimedia)
- Videoconferencia
  - ✓ Zoom, Microsoft Teams, Google Meet, Webex
- Juegos en línea multijugador
  - ✓ Fortnite, League of Legends, Call of Duty (transmiten audio y video en tiempo real)

2.7. Aplicaciones Multimedia

Streaming multimedia: El vídeo se transmite y reproduce **mientras se descarga**, sin esperar a tener el archivo completo.

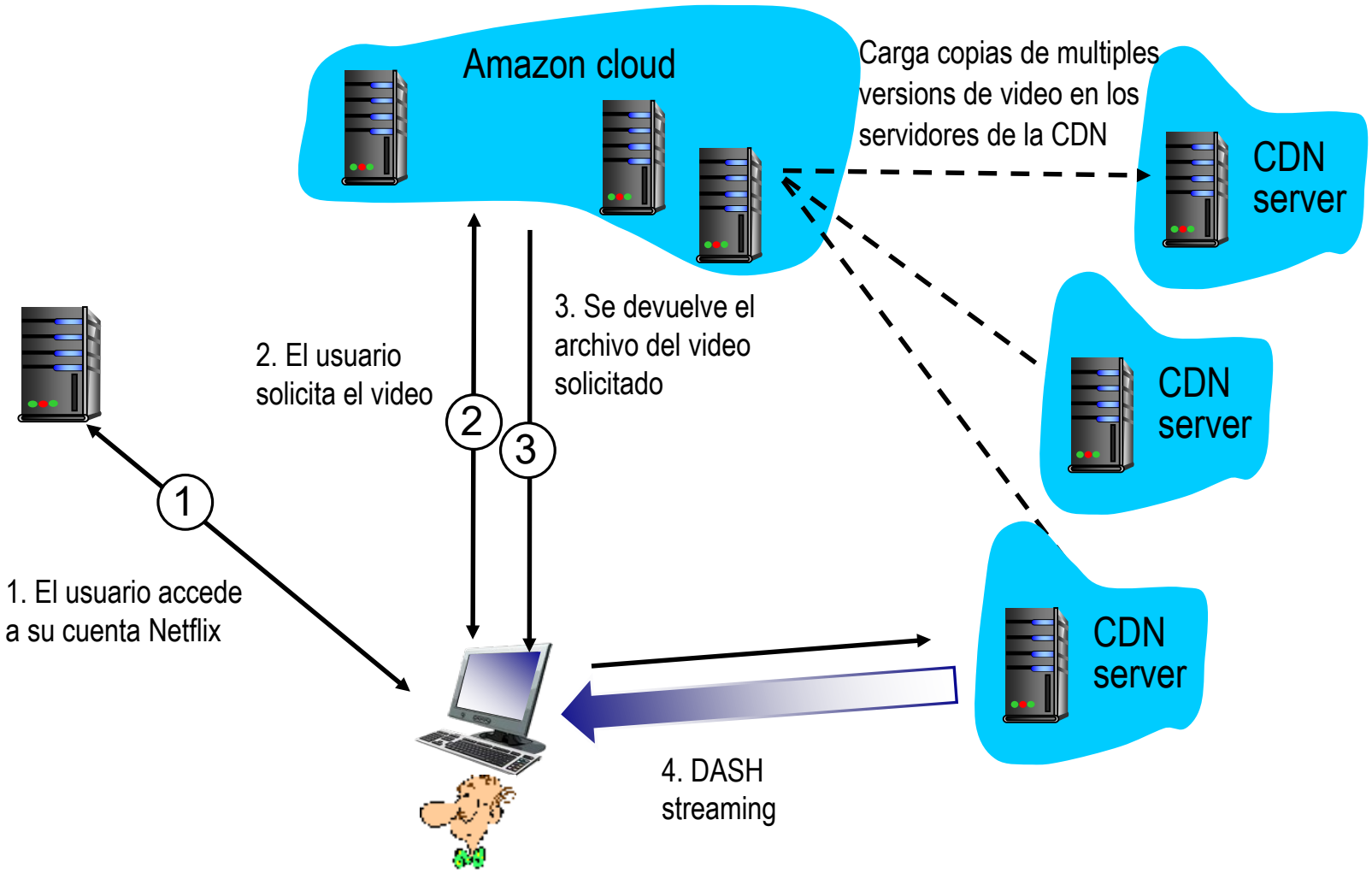
- **Streaming en vivo (live streaming):**
  - ✓ Transmisión en **tiempo real**
  - ✓ Ejemplos: Twitch (directo), YouTube Live, videoconferencias (Zoom, Teams)
  - ✓ Usa UDP (WebRTC, RTP) o QUIC (basado en UDP pero con mecanismos propios de fiabilidad)
  - ✓ Baja latencia es crítica
- **Streaming bajo demanda / VOD (Video on Demand):**
  - ✓ Video **pregrabado** pero transmitido como flujo
  - ✓ Ejemplos: Netflix, YouTube (videos subidos), Amazon Prime, Disney+
  - ✓ Usa TCP/HTTP/2 o QUIC/HTTP/3 con DASH
  - ✓ Usa buffering para compensar latencia
  - ✓ **No descarga el archivo completo** al dispositivo (se borra del buffer)
  - ✓ CDN + DASH = tecnologías para hacer streaming VOD más eficiente

2.7. Aplicaciones Multimedia

- Flujos dinámicos adaptativos sobre HTTP (DASH)
  - ✓ El vídeo se almacena en el servidor HTTP como un archivo normal con un url específico
  - ✓ Se codifica en versiones diferentes, con distintos niveles de calidad
  - ✓ Adaptación dinámica según ancho de banda disponible
  - ✓ Netflix, YouTube, Amazon Prime
- Redes de distribución de contenido (CDN)
  - ✓ Distribuir cantidades masivas de datos de vídeo a usuarios dispersos por todo el mundo
  - ✓ Una CDN:
    - gestiona servidores situados en múltiples ubicaciones geográficamente distribuidas
    - almacena copias de los videos (u otro contenido) en servidores cercanos al usuario
    - dirige cada solicitud de usuario a una ubicación de la CDN que proporcione mejor experiencia de usuario posible (menor latencia, mayor velocidad)

2.7. Aplicaciones multimedia

- Caso: Flujo de vídeo de Netflix



Fuente: Powerpoints slides copyright 1996-2016 J.F. Kurose, K.W. Ross